

# ScanIT - Team01

---

## ScanIT - Team01 (*student proposal*)

---

**Assessment:** ♦ Fit with adjustments · **Category:** Social Media · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

**Proposal (as submitted):** A native Android shopping assistant that identifies products, compares current prices across retailers, and suggests alternatives from vague requirements. Users scan a barcode/label or search conversationally (e.g. "small black headphones with strong bass under \$150"), refine by budget/colour/size/features/brand, and contribute updated prices, availability and product info — a community-driven price database with review/verification before data becomes trusted. Kotlin provides the native Android shell/UI; the performance-critical search & recommendation core is C++ via JNI. Named features: camera-assisted product identification (barcode + OCR); conversational hybrid search (intent extraction, remembered preferences, follow-up questions); a real-time C++ recommendation engine (filter → compare → rank); and a CRUD/crowdsourced backend with multi-store price comparison. Custom tool: an ImGui interface to monitor/test components, change search parameters, trigger searches, view API responses and debug the backend. Focus is current prices (not historical tracking), with explanations of why each result matches.

**Category note:** filed under **Social Media** — the load-bearing C++ piece is a ranking/recommendation service over community-contributed content with moderation, which is the Social Media pool's spine (matching/ranking, search & discovery, moderation, Firebase). Runner-up category: **CMS** (the crowdsourced contribute → verify → trusted-data workflow), which lacks a ranking/recommendation goal and so fits less well.

### Adjustments required for approval:

- **Clear the 2D gate with a genuinely custom-rendered data-display surface.** A native Kotlin app built purely from Android widgets/RecyclerViews would collapse into ordinary CRUD and fail the module's 2D requirement. Render at least the price-comparison / savings-breakdown

view (retailer cards, savings pills, a simple price bar/plot) in **OpenGL ES** inside a **GLSurfaceView**, drawn by the C++/render layer — not stock widgets.

- **The recommendation engine must be the team's own load-bearing C++ service.** The filter → compare → rank → "explain the match" pipeline (and the ranking behind conversational refinement) must be benchmarked C++ the team wrote. A library/model may extract intents from the conversational query, but an external "AI"/LLM API must **not** be the engine that does the matching/ranking (category guard-rail: the backend cannot be someone else's service).
- **Be realistic about live retailer prices.** Real multi-retailer price APIs are mostly unavailable or paid, and scraping often violates terms of service. Base the project on the **crowdsourced database + a seeded/mock (or legitimately available) retailer dataset**; treat any live feed as a bonus, not a dependency.

#### *References:*

- [ShopSavvy — Wikipedia](#) (barcode-scanning shopping app: finds online & local stores, compares prices, shows reviews/deals)
- [PriceRunner — Wikipedia](#) (price-comparison service matching retailer prices against a backend DB via fuzzy auto-matching + manual admin verification)
- [Google Lens — Wikipedia](#) (camera-assisted product identification: reads barcodes/QR/labels and returns product & shopping results)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | ScanIT |
|-------------------------------------------------------------|--------|
| External Database                                           | ✓      |
| Authentication »                                            | ✓      |
| Analytics »                                                 | ●      |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓      |
| Application Communication                                   | ✓      |
| Continuous Integration                                      | ✓      |
| Data-Driven Architecture                                    | ✓      |
| Front-end + Back-end                                        | ✓      |
| Custom Tools Development                                    | ✓      |
| 2D Graphical Output                                         | ♦      |
| 3D (strong students only)                                   | ✕      |
| Networking System                                           | ✓      |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile — the project is a native Android app (Kotlin UI + the C++ core via JNI), camera-first and touch-first (device: Pixel 9a); the Native PC build is the development/debug and custom-tools vehicle (no web target this trimester). **Extended:** Firebase + Linux server set up by M2 (full use T2).

**Front end:** Scanner screen (camera), conversational/multimodal search bar, refine chips, and the multi-store price-comparison sheet (retailer cards, stock badges, savings pills) with a custom-rendered price/savings view; plus the crowdsourced contribute/verify screens (OpenGL ES for the custom data-

display surface). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES) for the PC dev/debug build; Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

**Back end:** The C++ recommendation service (filter → compare → rank + match-explanation), the conversational-query intent/preference model, a search/index over the product catalog, external-price API adapters and the timed refresh loop, plus the crowdsourced CRUD store with a moderation/verification pipeline. **Languages:** C++.

**Backend scope:** Local now, server-ready (T2) — Firebase for the crowdsourced DB; the Linux server seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing piece is a **performant C++ service** — it parses the query (budget, product type, attributes), filters the catalog, compares candidates, ranks the best matches and decides when to ask a follow-up, plus a timed query loop that refreshes results/prices when data changes. This is genuine C++ information-retrieval work, not a database `where` clause or an external API call.
- *Data-driven:* products, variants, attributes, retailers, stock/price entries, matching weights and the conversational intent schema are all hot-reloadable JSON — no hard-coded catalog or ranking constants.
- *2D:* the price-comparison / savings view is **custom-rendered in OpenGL ES** (retailer cards, stock badges, savings pills, a price bar) — the domain-adapted 2D surface (see adjustment 1). *3D:* not needed. *Networking:* external retailer/price API calls + the crowdsourced-DB sync and (optional) push when a watched price changes.
- *Custom tool:* the **ImGui monitoring/testing console** — change search parameters, trigger searches, view API responses, watch system performance and debug the backend (PC-side); it doubles as the moderation/verification review surface.
- *External DB:* Firebase for the crowdsourced product/store/price database. *Auth* »: accounts for contributions and trust/attribution (safety-relevant for moderation). *Analytics* »: optional — which queries/products convert, to tune the ranker.

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** On the PC dev/debug build, C++17/20 with a fixed-timestep service loop driving the timed query/refresh pass and the recommendation pipeline; SDL2/3 for the dev harness, CMake, Git.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G9 recommendation engine and the timed refresh loop run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with products, variants, attributes, retailers, price/stock entries, matching weights and the intent schema in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1(loop & update hooks).
- **Feeds into:** M1G9 (scoring reads this model), M1G8 (Firebase syncs it), M2G11 (search index), M2G2 (the console edits it).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the ranker and query pipeline, Tracy profiling, Catch2 stress tests ranking a large synthetic catalog, GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the latency budget M2G1 validates on-device.

- **G9 Matching / Ranking Engine — Foundation** — the load-bearing C++ recommender: filter → compare → rank products against the parsed requirements.

Tech & dependencies

- **Tech:** Priority queues and weighted scoring over product attributes (price, type, features), hard-constraint filtering (budget/compatibility), incremental re-ranking as preferences change (custom C++) — not a database query.
  - **Prerequisites:** G2 (product/attribute model), G1 (tick).
  - **Feeds into:** M2G8 (recommendations II + match-explanation), M2G11 (search), the comparison sheet.
- **G6 Custom 2D UI Rendering** — the multi-store price-comparison / savings view rendered custom (satisfies the 2D gate).

Tech & dependencies

- **Tech: Web build:** SDL2/3 renderer or OpenGL ES, GLSL, FreeType (text), stb\_image for the retailer cards, stock badges, savings pills and a price bar on the PC dev build. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** G1 (shell & loop).
  - **Feeds into:** The price-comparison sheet; M2G6 moderation views reuse the widget layer.
- **G8 Firebase Integration — Identity & Data** — the crowdsourced product/store/price database (CRUD) and accounts.

Tech & dependencies

- **Tech:** Firebase Auth, Firestore/Realtime DB for products/stores/prices/contributions, REST API from C++ and the JS/Kotlin SDK on the front-end.
- **Prerequisites:** G2 (data model to sync).
- **Feeds into:** M2G6 (moderation pipeline), M2G8 (reputation/trust as a ranking signal).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

## Tech & dependencies

- **Tech: Web build:** Tracy profiling of ranking/search, ASan on MSVC, memory pools for the catalog/index, and validation on the PC dev build; Linux server set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI; validate ranking/search latency and the GLSurfaceView comparison view on the Pixel 9a.
  - **Prerequisites:** All M1 goals (hardens the M1 codebase).
  - **Feeds into:** The native Android deliverable and headroom for large catalogs on-device.
- **G2 ★ Content Editor — Core** — the ImGui monitoring/testing console + the moderation/verification review surface: change search params, trigger searches, view API responses, edit catalog data, action contributions.

## Tech & dependencies

- **Tech:** C++ + Dear ImGui, JSON serialization, undo/redo command stack; panels to tune matching weights, inspect API responses and review/verify/correct crowdsourced entries.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live result preview).
- **Prerequisites:** G2 (data model), M1G9 (engine to drive/inspect).
- **Feeds into:** M2G8 (weights it tunes), M2G6 (moderation actions applied through it).

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

## Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, packaging of the PC dev build artifact via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI) packaging of the Android APK/AAB; OpenGL ES in a GLSurfaceView where realtime 2D is needed — no SDL on native.
  - **Prerequisites:** G3 (CI and test foundation).
  - **Feeds into:** Release/showcase builds of every ScanIT goal.
- **G8 Ranking / Matching Engine II** — recommendations + "why it matches": conversational refinement, alternatives and explanations.

#### Tech & dependencies

- **Tech:** Scoring models (weighted features, cosine similarity over attribute/feature vectors), candidate generation, evaluation metrics, and a match-explanation trace; reputation/trust of a price entry as a ranking signal.
  - **Prerequisites:** M1G9 (ranking foundation), M1G8 (data).
  - **Feeds into:** The recommendation feed, alternatives, and the "why this matches" explanation.
- **G11 Search & Discovery Engine** — the conversational/hybrid search: parse the query, index the catalog, retrieve candidates.

#### Tech & dependencies

- **Tech:** Incremental inverted index, TF-IDF ranking and a custom query parser (C++) that extracts budget/type/attributes from natural language and feeds the ranker; barcode/OCR results enter the same pipeline.
  - **Prerequisites:** M1G2 (content model), M1G9 (ranker).
  - **Feeds into:** The search bar, refine flow and follow-up questions.
- **G6 Content Moderation Dashboard & Pipeline** — the crowdsourced review/verify/correct/report workflow that turns contributions into trusted data.

#### Tech & dependencies

- **Tech:** Validation filters (C++), a contribution/report queue in Firestore, and a dashboard (ImGui or front-end) for community/admin review, verify, correct and reject.
- **Prerequisites:** M1G2 (content model), M1G8 (Firebase identity/data).
- **Feeds into:** Trusted price/product data; M2G8 (trust as a ranking signal).

*(Proposed new pool goal (requires instructor approval):* **Camera Capture & OCR/Barcode Pipeline** — camera frame capture, barcode decode and label/model-number OCR feeding the shared search pipeline; the Social Media pool has no vision goal, so the camera-assisted identification feature would either be a new appended pool goal or folded into M1G2/M2G11. The three picks above stand regardless.)

## Track Goals & Team Composition

**Team composition (set by instructor):** 6 CS + 1 Design + 0 Art (7) — ScanIT is a UX-heavy utility app (scanner, conversational search + refine, the multi-store comparison sheet, the crowdsourced contribute/verify flows), and the visual surface (icons, store-logo placeholders, stock badges, savings pills) is UI-art the designer owns — there is no character/illustration production, so 0 artists (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see [team-formation.pdf](#)). With 0 artists, the designer takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality) — the last Design goal each milestone. *Note:* this is the **solo-designer + 0-artist** case the design pool flags — one designer now carries the full 4-goals-per-milestone Design + UI/UX load *and* the D11 art surface, which is a heavy load for a UX-heavy app; a 2nd designer would relieve it if the team can, and the instructor should confirm which conditional bends if the load is too tight.

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the scan → search → refine → compare → contribute product spec, the conversational + crowdsourced USP, scope sign-off)
- M1D2 Core-Loop Interactive Prototype (Figma flow: scan/search → refine chips → price-comparison sheet → contribute)
- M1D11 Art Proof of Concept (conditional — no artist: icon set, store-logo/badge visual language, UI style guide)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality (conditional — no artist: UI art/ icons implemented to standard in the build)

- **UI/UX M1** (graded by the UI/UX instructor):

- M1U1 ★ UI Standards & Wireframe Baseline — the scanner, search, comparison and contribute screens.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the required screens navigable unaided on the mobile target.

- M2U2 ★ UX Validation & Final Polished UI — validated, polished comparison sheet and search/refine flow.
- **Art:** 0 artists — the art surface is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2).
- **TEAM M1:**
  - M1TA1 ★ Overall Audio Experience — light UI/scan feedback SFX and optional subtle ambience from the week-5 needs list, present & working in the prototype (scan confirm, result ready, error), suitable and correctly triggered. Audio is not load-bearing here — a lecturer waiver on scope may apply.
  - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping UI SFX.
- **TEAM M2:**
  - M2TA1 ★ Overall Audio Experience — the standard floor tailored down for a utility app (scan/confirm/error SFX + optional ambience in the showcase build).
  - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing for stacked UI cues).
  - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the ImGui console), prototype demo, team post-mortem (wk14)

---

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*